



"tO PrOGRAM YOu wILL IEArN"



Resolución de Problemas y Algoritmos

Contenidos mínimos

Dra. Jessica A. Carballido
jessicarballido@gmail.com

Abril, 2024

DCIC (UNS), Bahía Blanca, Buenos Aires, Argentina.

Tabla de contenidos

1.	PROBLEMAS	4
1.1	<i>¿Qué es un problema?</i>	4
1.2	<i>Tipos de problemas</i>	4
1.3	<i>Tipos de problemas resueltos en RPA</i>	4
2	ALGORITMOS	1
2.1	<i>Algoritmos de la vida cotidiana</i>	1
2.2	<i>Algoritmos planteados en RPA: datos de entrada y datos de salida</i>	1
3	DATOS Y EXPRESIONES	4
3.1	<i>Datos</i>	4
3.2	<i>Expresiones</i>	5
4	ACCIONES: ESTRUCTURAS DE CONTROL	9
4.1	<i>Secuencia</i>	11
4.2	<i>Condicional, caminos alternativos de acción</i>	13
4.3	<i>Repetición</i>	20
5	PASCAL, SINTAXIS BÁSICA	¡ERROR! MARCADOR NO DEFINIDO.
6	PRIMITIVAS	¡ERROR! MARCADOR NO DEFINIDO.
7	PASCAL, FUNCIONES Y PROCEDIMIENTOS	¡ERROR! MARCADOR NO DEFINIDO.
8	RECURSIVIDAD	¡ERROR! MARCADOR NO DEFINIDO.
9	PASCAL, FUNCIONES RECURSIVAS	¡ERROR! MARCADOR NO DEFINIDO.
10	PASCAL, AMBIENTES DE REFERENCIAMIENTO	¡ERROR! MARCADOR NO DEFINIDO.

Índice de figuras

FIGURA 1. PROBLEMA: DISCREPANCIA ENTRE EL ESTADO INICIAL Y EL FINAL	4
FIGURA 2. NUESTRO ESQUEMA DE LAS PARTES PRINCIPALES DE UN ALGORITMO	2
FIGURA 3. ESQUEMA DEL ALGORITMO “ ENCONTRAR MENOR ” Y CASO PARTICULAR DE USO	3
FIGURA 4. DEL PROBLEMA AL PROGRAMA	3
FIGURA 5. ESQUEMA DEL ALGORITMO “ SON PARES ”, CASO PARTICULAR DE USO DEL ALGORITMO CON RESPUESTA NEGATIVA Y CASO PARTICULAR DE USO CON RESPUESTA ES AFIRMATIVA	4
FIGURA 6. OPERADORES ARITMÉTICOS MÁS USADOS	5
FIGURA 7. EJEMPLO DE DIVISIÓN ENTERA Y SUS PARTES	6
FIGURA 8. OPERADORES LÓGICOS	7
FIGURA 9. TABLA DE VERDAD DEL OPERADOR LÓGICO Y (AND)	7
FIGURA 10. TABLA DE VERDAD DEL OPERADOR LÓGICO O (OR)	7
FIGURA 11. TABLA DE VERDAD DEL OPERADOR LÓGICO NO (NOT)	7
FIGURA 12. OPERADORES RELACIONALES	8
FIGURA 13. PRECEDENCIA DE LOS OPERADORES ARITMÉTICOS	8
FIGURA 14. LA ASIGNACIÓN COMO UNA ACCIÓN EN DOS ETAPAS	9
FIGURA 15. ESTRUCTURA DE CONTROL CONDICIONAL	14
FIGURA 16. DIAGRAMA DE FLUJO DE LA ESTRUCTURA DE CONTROL CONDICIONAL	14
FIGURA 17. EJEMPLO DE ESTRUCTURA DE CONTROL CON UN CAMINO CONDICIONAL	14
FIGURA 18. EJEMPLO DE ESTRUCTURA DE CONTROL CON UN CAMINO CONDICIONAL: DIAGRAMA DE FLUJO	15
FIGURA 19. EJEMPLO DE ESTRUCTURA DE CONTROL CON DOS CAMINOS MUTUAMENTE EXCLUYENTES A PARTIR DE UNA CONDICIÓN	15
FIGURA 20. ESTRUCTURA DE CONTROL PARA MÚLTIPLES CAMINOS ALTERNATIVOS MUTUAMENTE EXCLUYENTES	17
FIGURA 21. ESTRUCTURA DE CONTROL REPETITIVA: MIENTRAS-HACER	20
FIGURA 22. DIAGRAMA DE FLUJO DE LA ESTRUCTURA DE CONTROL REPETITIVA: MIENTRAS-HACER	21
FIGURA 23. ESTRUCTURA DE CONTROL REPETITIVA: REPETIR-HASTA	24
FIGURA 24. DIAGRAMA DE FLUJO DE LA ESTRUCTURA DE CONTROL REPETITIVA: REPETIR-HASTA	24
FIGURA 25. ESTRUCTURA DE CONTROL REPETITIVA: PARA-DESDE-HASTA	26

I. PROBLEMAS

I.1 ¿QUÉ ES UN PROBLEMA?

Un problema es una situación, cuestión o desafío que requiere una solución o una respuesta. Por lo general, implica una discrepancia entre una situación inicial y una situación final deseada, y puede surgir en diversos contextos, como matemáticas, ciencia, tecnología, vida cotidiana, entre otros. Resolver un problema implica identificar y comprender la naturaleza del problema, desarrollar estrategias para abordarlo y aplicarlas de manera efectiva para alcanzar una solución satisfactoria.



	6			5	7				
	9		6	8					
7	8		5			4			
	1	5		4	2	9			
	7		3						
3	9				7	1			
	5		4		8		9		
	8	2	6						
				2	7	4		6	

1	3	6	4	9	2	5	8	7	
5	9	4	7	6	8	1	3	2	
7	2	8	3	1	5	9	6	4	
8	6	1	5	7	4	2	9	3	
2	7	5	9	3	1	6	4	8	
3	4	9	2	8	6	7	1	5	
6	5	7	1	4	3	8	2	9	
4	8	2	6	5	9	3	7	1	
9	1	3	8	2	7	4	5	6	

Figura 1. Problema: discrepancia entre el estado inicial y el final

Ejemplos:

- Encontrar el camino más corto desde casa hasta la universidad
- Ganar un partido de damas
- Aprobar una materia
- Armar un rompecabezas
- Encontrar el valor de una variable en una ecuación
- Calcular el perímetro de un terreno

I.2 TIPOS DE PROBLEMAS

Los problemas se pueden clasificar de acuerdo a numerosos criterios. En nuestro contexto, nos sirve ordenarlos según la cantidad de soluciones que tienen.

Con una única solución:

- Sumar los números del 1 al 100
- Calcular el perímetro de un cuadrado a partir de la longitud de uno de sus lados

Con varias soluciones:

- Nombrar a tres actores argentinos
- Preparar una torta

Sin solución o con infinitas soluciones

- Calcular la división de cualquier número por cero
- $x * y = 0$
- Obtener la raíz cuadrada de un número negativo

I.3 TIPOS DE PROBLEMAS RESUELTOS EN RPA

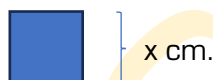
En esta materia solamente resolveremos **problemas con una única solución**. Sin embargo, si bien la solución buscada estará clara, en general no existe una forma única de obtenerla. Ciertamente, no hay un único enfoque para resolver problemas mediante algoritmos. Por esta razón es fundamental estudiar las técnicas de manera progresiva, empezando gradualmente por lo más simple y avanzando hacia lo más complejo.

Otro aspecto importante es que los problemas que buscamos resolver en esta materia tienen que ser **problemas generales**. Veámoslo con un ejemplo sencillo. Obtener el perímetro de un cuadrado cuya longitud de lado es 2 cm. es un problema que tiene una única solución, pero no es un problema general. Este problema, si bien tiene una solución que es 8 cm., no es general ya que al resolverlo estamos respondiendo a una **pregunta particular**, es decir estamos resolviendo una instancia particular de un problema. La definición general de este problema va a expresarse de la siguiente forma: *obtener el perímetro de un cuadrado a partir de la longitud de uno de sus lados (representada por ejemplo con x)*. Este enunciado es un enunciado típico que podremos resolver en esta materia y define **una familia de problemas**. Es general, y al resolverlo tiene una solución única.

Problema particular

Perímetro: $2 * 4 = 8$ cm.

Problema general

Perímetro: $x * 4$ cm.

☛ Si te propongo calcular el 20% de 500, ¿es un problema con una única solución? ¿Es general? ¿Podrías cambiar el enunciado para representar una familia de problemas, que contenga a la instancia particular que te propuse? Escribí abajo tus ideas, y cuando quieras me mostrarás lo que pensaste.



Autor:
Dr. Diego C. Martínez
Director Decano del DCIC

2 ALGORITMOS

2.1 ALGORITMOS DE LA VIDA COTIDIANA

En RPA hallar la solución a nuestro problema va a consistir en encontrar una secuencia de pasos que permitan pasar del estado inicial a un estado final (en el que ya tenemos la solución al problema) respetando algunas restricciones. A esta secuencia de pasos se la denomina algoritmo¹.

Los algoritmos están presentes en muchas actividades de la vida cotidiana. A continuación, algunos ejemplos de algoritmos que seguimos en nuestra vida cotidiana:

- Receta de cocina: Una receta es esencialmente un algoritmo que describe paso a paso cómo preparar un plato específico. Incluye una lista de ingredientes y una serie de instrucciones detalladas sobre cómo combinar y cocinar esos ingredientes para obtener el resultado deseado.
- Instrucciones para conducir: Al aprender a conducir, seguimos un conjunto de instrucciones específicas que constituyen un algoritmo para manejar un automóvil de manera segura. Esto puede incluir cómo encender el motor, ajustar los espejos, cambiar de marcha, señalar al girar y obedecer las señales de tráfico.
- Lavado de ropa: El proceso de lavar la ropa también sigue un algoritmo. Incluye pasos como separar la ropa por colores y tipos de tejido, seleccionar el ciclo y la temperatura adecuados en la lavadora, agregar detergente, etc.
- Hacer ejercicio: Cuando seguimos un plan de ejercicios, estamos siguiendo un algoritmo que especifica qué ejercicios realizar, cuántas repeticiones hacer, durante cuánto tiempo hacer cada ejercicio, etc.
- Ir de compras: Al hacer la lista de compras y luego ir al supermercado, seguimos un algoritmo. La lista de compras especifica los artículos que necesitamos comprar, y luego seguimos un proceso para encontrar esos artículos en la tienda, ponerlos en el carrito, pagar por ellos, etc.
- Rutina matutina: Muchas personas siguen una rutina matutina que puede considerarse un algoritmo. Esto incluye pasos como despertarse a una hora específica, ducharse, vestirse, desayunar, cepillarse los dientes, etc.

En resumen, los algoritmos son instrucciones paso a paso para realizar una tarea específica. En esta materia buscaremos **diseñar algoritmos** para resolver problemas de pequeña escala e **implementarlos en programas** escritos en el lenguaje de programación Pascal, cumpliendo con los siguientes criterios de calidad:

- Generales
- Eficientes
- Eficaces
- Finitos

2.2 ALGORITMOS PLANTEADOS EN RPA: DATOS DE ENTRADA Y DATOS DE SALIDA

Hablando informalmente, un algoritmo es la descripción precisa de los pasos que nos llevan a la solución del problema planteado. Estos pasos son, en general, acciones u operaciones que se efectúan sobre ciertos objetos. La descripción de un algoritmo está siempre compuesta por tres partes: entrada [datos], acciones [instrucciones para llevar a cabo el proceso] y salida [resultados].

¹ La palabra algoritmo se deriva de la traducción al latín de la palabra árabe *alkhowarizmi*, nombre de un matemático y astrónomo árabe que escribió un tratado sobre manipulación de números y ecuaciones en el siglo IX.

En este sentido, un algoritmo se puede comparar de manera muy simplificada y gráfica con una función matemática:

$$\begin{array}{ccccccc}
 + & : & ZZ \times ZZ & \longrightarrow & ZZ \\
 \text{[algoritmo]} & & \text{[entrada]} & \text{[acciones]} & \text{[salida]}
 \end{array}$$

En el ejemplo estamos dando información sobre la suma, que recibe dos datos enteros y luego de sumarlos [con las acciones] se obtiene un dato entero.

En este sentido, incluso en algoritmos no matemáticos se pueden identificar las tres partes: entrada, acciones y salida. Veamos algunos ejemplos de los tipos de problemas que vamos a resolver en RPA:

- A partir de una fecha, encontrar la fecha siguiente.
- A partir de un valor de radio en centímetros, calcular el área y el perímetro de una circunferencia. 🌀
- A partir de dos números, obtener el mayor.
- A partir de tres números, obtener el menor.
- A partir del costo de un producto en pesos, una cantidad de compra y un criterio para el descuento, calcular el precio final.
- A partir de dos valores, calcular la suma de todos los números contenidos en el intervalo definido por ellos. 🌀
- A partir de dos números, determinar si ambos son números pares.

Todo algoritmo consta de tres secciones principales: **entrada**, son los datos que se introducen para ser transformados; **acciones**, es el conjunto de operaciones para dar solución al problema, y **salida**, son los resultados obtenidos a través de las acciones. En este contexto entonces, llamamos datos de entrada a la información con la que contamos, y datos de salida a la información que queremos obtener. En general vamos a representar esto de la siguiente manera:



Figura 2. Nuestro esquema de las partes principales de un algoritmo

A lo largo de este texto vamos a retomar varias veces esta imagen, para ilustrar la importancia de detectar esta información antes de empezar a escribir un algoritmo. Por ejemplo, si tomamos el problema “A partir de tres números naturales, obtener el menor”, podemos determinar que los datos de entrada serán tres números naturales, y el dato de salida también será un número (aquel que cumpla con la propiedad de ser el más chico de los tres ingresados). A este problema lo llamaremos “encontrarMenor” y podría ser representado de la siguiente manera:

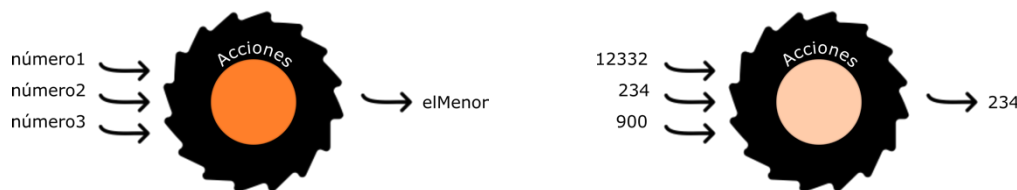


Figura 3. Esquema del algoritmo "encontrarMenor" y caso particular de uso

Nuestro objetivo en esta materia será plantear inicialmente distintos problemas simples y resolverlos mediante algoritmos. En un paso siguiente estos algoritmos serán traducidos al lenguaje de programación Pascal. Este lenguaje es el que nos permite comunicarnos directamente con la computadora. Más adelante veremos en detalle este tema.

*Figura 4. Del problema al programa*

✎ Para los ítems marcados anteriormente, armá abajo los esquemas correspondientes al algoritmo general, y a dos casos particulares de uso.

3 DATOS Y EXPRESIONES

3.1 DATOS

En los algoritmos, se manipulan y transforman **datos** a través de las acciones. Los datos de entrada representan el estado inicial y son utilizados y/o transformados mediante las acciones para así obtener los datos de salida, y en estos datos de salida se encuentra representada la respuesta o solución buscada. En esta materia trabajaremos con problemas que se representan mediante **datos pertenecientes a los dominios numéricos, caracteres y lógicos**. Los caracteres son las letras minúsculas y mayúsculas, los dígitos y cualquier símbolo que podemos obtener con una tecla del teclado de la computadora. Los datos lógicos son los que tienen valor verdadero o falso. A su vez, los datos del dominio de los números se distinguirán entre enteros y reales (con coma). En el ejemplo de la figura 3, podemos especificar que los datos de entrada son tres números enteros, y que el dato de salida es un número entero.

Si tomamos el problema de la sección anterior “A partir de dos números, determinar si ambos son números pares”, tenemos un ejemplo donde hay dos datos de entrada de tipo entero y un dato de salida que es lógico, ya que debemos responder a una pregunta por sí o por no. A este algoritmo lo llamaremos “sonPares” y el esquema es el siguiente:

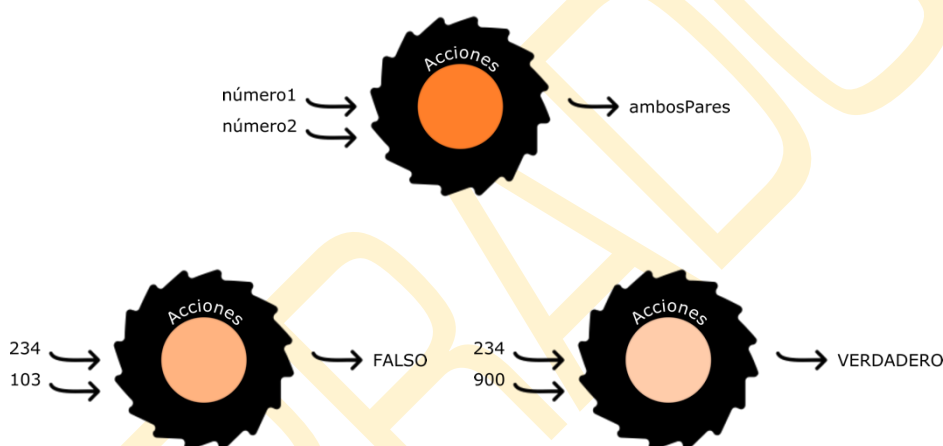


Figura 5. Esquema del algoritmo “sonPares”, caso particular de uso del algoritmo con respuesta negativa y caso particular de uso con respuesta es afirmativa.

En este punto podemos empezar a refinar el proceso de diseño de nuestros algoritmos. Como vimos, el primer paso debería ser siempre armar el esquema general como los vistos anteriormente, donde se visualice claramente lo que recibe (**DE: Datos de entrada**) y lo que devuelve el algoritmo (**DS: Datos de salida**). Estos datos pertenecen a un dominio, con lo que vamos a escribir una primera versión del algoritmo con toda esta información, de la siguiente manera:

```

Algoritmo sonPares
DE: número1 (entero), número2 (entero)
DS: ambosPares (lógico)
... acciones

```

Al “lenguaje” que usamos para escribir nuestros algoritmos se lo llama pseudo-código. Trataremos de respetar todas las partes del mismo, el título, la sección de datos de entrada y de salida, y luego las acciones. De la misma manera, podemos escribir una primera versión incompleta en pseudo-código del algoritmo “encontrarMenor”:

Algoritmo encontrarMenor

DE: número1 (entero), número2 (entero), número3 (entero)

DS: elMenor (entero)

... acciones

3.2 EXPRESIONES

Los datos, que pertenecen a distintos dominios (tipos), se combinan mediante expresiones. Una expresión es una combinación de valores, nombres de datos, operadores y funciones que se evalúan para producir un resultado. Estas expresiones pueden ser aritméticas, lógicas, o incluso de caracteres (veremos ejemplos más adelante), dependiendo de su propósito.

- **Aritméticas:** Los operandos que intervienen son numéricos, el resultado es numérico y los operadores y funciones que se aplican son aritméticos.

Ejemplo: $5 + 10 * 2$

5, 10 y 2 son operandos (los valores con los cuales se va a calcular).

+ y * son operadores (especifican las operaciones).

Es una expresión aritmética ya que su resultado es un número.

UNA "CUENTA"

- **Lógicas:** El resultado es VERDADERO o FALSO. Se construyen mediante los operadores relacionales y lógicos, y pueden tener en su interior sub-expresiones aritméticas.

Ejemplo: $3 > 5$

3 y 5 son operandos (los valores que se usarán).

> es un operador relacional (especifica la operación).

Es una expresión lógica ya que devuelve un valor lógico.

UNA "PREGUNTA"
CON RESPUESTA
SI O NO

True
False

En resumen, en programación, una expresión es una combinación de elementos que se evalúan para producir un resultado.

3.2.1 EXPRESIONES ARITMÉTICAS

Los protagonistas en las expresiones aritméticas son los operadores aritméticos. Además, también pueden intervenir distintas funciones matemáticas para realizar cálculos adicionales. Veamos a continuación los operadores aritméticos más usados:

Operadores aritméticos en pseudocódigo:	
+	Suma
-	Resta
*	Multiplicación
**	Potencia (también se utiliza el carácter <i>flecha arriba</i> ↑ o el carácter <i>acento circunflejo</i> ^)
/	División real
div	División entera (también se utiliza el carácter <i>barra invertida</i> \)
mod	Módulo (resto de la división entera)
+	Signo más
-	Signo menos

© carlospes.com

Figura 6. Operadores aritméticos más usados

Es imprescindible en esta materia tener claro el funcionamiento de la **división entera**. La división entera es una operación aritmética en la que se divide un número ENTERO por otro número ENTERO y se obtiene un cociente entero y un resto, sin tener en cuenta ningún decimal. Es decir,

se calcula cuántas veces cabe un número entero en otro, sin considerar las fracciones. El resto es lo que sobra.

Ejemplos: Si tenemos 10 caramelos y queremos repartirlos entre 3 amigos de manera equitativa, la división entera nos indica cuántos caramelos recibirá cada amigo y cuántos caramelos sobran. En este caso, 10 dividido por 3 es igual a 3 (3 caramelos a cada uno de los 3 amigos), y sobra 1 caramelo que no se puede repartir.

En resumen, la división entera devuelve el cociente entero de una división, sin incluirse nunca decimales.

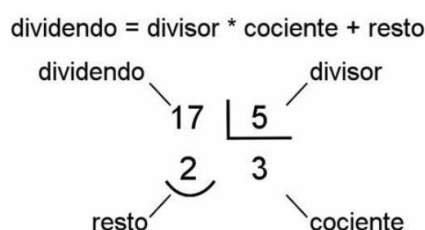


Figura 7. Ejemplo de división entera y sus partes

Ejemplos de expresiones aritméticas y sus correspondientes resultados:

11.2 + 4 div 3	→ 11.2 + 1 → 12.2
3 * 15 mod 4	→ 45 mod 4 → 1
9 - 3 * 5 + 1	→ 9 - 15 + 1 → -5
2*5 + 11/2	→ 10 + 5.5 → 15.5
2.6 div 2	→ ERROR: no se puede hacer división entera (dividendo real)
10 div 3.1	→ ERROR: no se puede hacer división entera (divisor real)

3.2.2 EXPRESIONES LÓGICAS

Los protagonistas en las expresiones lógicas (o también llamadas booleanas) son los operadores lógicos y los operadores relacionales. Como resultado de una expresión lógica siempre obtendremos un valor lógico, es decir VERDADERO o FALSO. Se puede pensar como una pregunta a la que respondemos con sí o con no.

Operadores lógicos

Los operadores lógicos son herramientas usadas tanto en programación como en matemáticas para realizar operaciones lógicas relacionando valores o expresiones booleanas. Estos operadores permiten combinar múltiples condiciones, y a la vez evaluar si se cumplen ciertas propiedades lógicas.

Existen tres operadores lógicos principales:

- **AND (Y lógico):** El operador AND devuelve verdadero ('true') si y solo si las dos condiciones que se combinan con él son verdaderas. En otras palabras, si las dos expresiones que se están evaluando son verdaderas, el resultado de la operación AND será verdadero.
- **OR (O lógico):** El operador OR devuelve verdadero si al menos una de las condiciones unidas por él es verdadera. En otras palabras, si al menos una de las expresiones evaluadas es verdadera, el resultado de la operación OR será verdadero.
- **NOT (Negación lógica):** El operador NOT devuelve el valor de verdad opuesto al valor de la condición asociada. Si la expresión es verdadera, el operador NOT devuelve falso, y si la expresión es falsa, el operador NOT devuelve verdadero.

Estos operadores son fundamentales para construir expresiones complejas en la programación, permitiendo tomar decisiones basadas en múltiples condiciones. ¡Asegurate de comprender bien estos conceptos antes de seguir avanzando!

Operadores lógicos en pseudocódigo:	
y	Conjunción
o	Disyunción
no	Negación

Figura 8. Operadores lógicos

<expresión_1>	<expresión_2>	<expresión_1> y <expresión_2>
verdadero	verdadero	verdadero
verdadero	falso	falso
falso	verdadero	falso
falso	falso	falso

Figura 9. Tabla de verdad del operador lógico Y (AND)

<expresión_1>	<expresión_2>	<expresión_1> o <expresión_2>
verdadero	verdadero	verdadero
verdadero	falso	verdadero
falso	verdadero	verdadero
falso	falso	falso

Figura 10. Tabla de verdad del operador lógico O (OR)

<expresión>	no <expresión>
verdadero	falso
falso	verdadero

Figura 11. Tabla de verdad del operador lógico NO (NOT)

Operadores relacionales

Los operadores relacionales permiten realizar comparaciones principalmente entre valores numéricos o expresiones aritméticas. Al aplicarlos se verifica si la comparación se cumple o no, es decir, el resultado de aplicarlos es un valor de verdadero o falso.

Los operadores relacionales más comunes son:

- Igual a (=): Este operador verifica si dos valores son iguales. Si los valores son iguales, la expresión es verdadera; de lo contrario, es falsa.
- Diferente de (<>): Este operador verifica si dos valores son diferentes. Si los valores son diferentes, la expresión es verdadera; de lo contrario, es falsa.
- Mayor que (>): Este operador verifica si el primer valor es mayor que el segundo valor. Si el primer valor es mayor, la expresión es verdadera; de lo contrario, es falsa.
- Menor que (<): Este operador verifica si el primer valor es menor que el segundo valor. Si el primer valor es menor, la expresión es verdadera; de lo contrario, es falsa.
- Mayor o igual que (>=): Este operador verifica si el primer valor es mayor o igual que el segundo valor. Si el primer valor es mayor o igual, la expresión es verdadera; de lo contrario, es falsa.

- Menor o igual que [$\leq$]: Este operador verifica si el primer valor es menor o igual que el segundo valor. Si el primer valor es menor o igual, la expresión es verdadera; de lo contrario, es falsa.

Operadores relacionales en pseudocódigo:	
<	Menor que
<=	Menor o igual que
>	Mayor que
>=	Mayor o igual que
=	Igual que
<>	Distinto que

Figura 12. Operadores relacionales

Precedencia de operadores

Para poder evaluar correctamente las expresiones, siempre es necesario conocer las reglas de precedencia de operadores. Esto es, cuando hay más de un operando en la misma expresión, ¿cuál debo evaluar primero? La precedencia en el pseudocódigo para expresiones aritméticas suele ser como la definida para las expresiones matemáticas. Sin embargo, al llegar el momento de programar, cada lenguaje puede definir su propia tabla de precedencia de operadores. Por esta razón es importante que si empezamos a programar con un lenguaje nuevo, consultemos las reglas de precedencia para no obtener resultados inesperados.

Prioridad de los operadores aritméticos (de mayor a menor) en pseudocódigo:	
+ -	Signos más y menos
**	Potencia
* / div mod	Multiplicación, división real, división entera y módulo
+ -	Suma y resta

Figura 13. Precedencia de los operadores aritméticos

Ejemplos de expresiones lógicas y sus correspondientes resultados:

(8 > 3)	➔ VERDADERO
NO (30=45)	➔ VERDADERO
(1>10) AND (3=3)	➔ FALSO
(51<50) OR (300<10)	➔ VERDADERO

- ☛ Para las siguientes expresiones, pinta el recuadro con celeste si es aritmética o con amarillo si es lógica. En el siguiente recuadro escribí el resultado que se obtiene de evaluarla.

(5 + 2) * (10 - 3.3)	
(8 > 3) or (2 < 10)	
(5 == 5) and (10 < 7)	
20 div (3 + 2)	
55 <= ((15 mod 4) + (7 div 2))	
(2 * (3 + 1.5) - (7 div 2)) = 0	
(7 >= 7) and (4 <= 3)	
not ((11 div 2) == 5)	

Estos ejemplos muestran cómo podemos combinar sub-expresiones aritméticas y lógicas dentro de expresiones más grandes para realizar cálculos complejos y evaluar condiciones elaboradas.

4 ACCIONES: ESTRUCTURAS DE CONTROL

Las acciones en nuestros algoritmos son las que permiten obtener los datos de salida a partir de la información contenida en los datos de entrada. La principal **acción** en la que se basan todos los lenguajes de programación es **LA ASIGNACIÓN**. La asignación nos permite dar un valor específico al nombre de un dato. Lo escribiremos de la siguiente manera:

Nombre $\leftarrow$ valor

Mediante esta acción estamos especificando que luego de que se lleve a cabo, el dato "Nombre" tendrá el valor "valor". Veamos más ejemplos de asignaciones:

- $a \leftarrow 10$ Luego de "ejecutar" esta acción, el dato "a" vale 10.
- $b \leftarrow (10 = 0)$ Luego de "ejecutar" esta acción, el dato "b" vale falso.
- $c \leftarrow 12 \text{ div } 5 + 1$ Luego de "ejecutar" esta acción, el dato "c" vale 3.

Como podemos observar, del lado derecho de la flecha, puede aparecer tanto un valor literal (la constante 10 por ejemplo) como una expresión. Dicha expresión, llegado el momento de ejecutar la asignación, será evaluada primero para obtener un resultado, y ese resultado será el valor que se asigne luego al nombre del dato que aparece en el lado izquierdo de la flecha. Por este motivo podemos pensar en la asignación como una acción que se lleva a cabo en dos etapas:

Etapas 1: evaluar la expresión que aparece del lado derecho de la flecha (si del lado derecho de la flecha tenemos un valor constante, esto es trivial).

Etapas 2: asignar el valor obtenido de dicha evaluación al nombre del dato que está a la izquierda de la flecha.

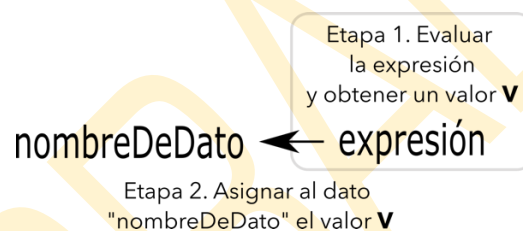


Figura 14. La asignación como una acción en dos etapas

Volvamos a los tres ejemplos anteriores y analicemos cómo funcionan estas dos etapas en cada uno de ellos:

- $a \leftarrow 10$

Miramos lo que hay del lado derecho de la flecha: tenemos un valor constante (literal) 10. No tenemos que evaluar nada, ese 10 es el que se asigna al dato "a" que aparece del lado izquierdo de la flecha.

- $b \leftarrow (10 = 0)$

Miramos lo que hay del lado derecho de la flecha, tenemos una expresión lógica $(10=0)$. Evaluamos la expresión y obtenemos un falso ya que 10 no es igual a 0. Ese falso se asigna al dato "b" que aparece a la izquierda de la flecha.

- $c \leftarrow 12 \text{ div } 5 + 1$

Miramos lo que hay del lado derecho de la flecha, tenemos una expresión aritmética $(12 \text{ div } 5 + 1)$. Para evaluar esa expresión, la precedencia de los operadores nos indica que primero debemos calcular la división entera $(12 \text{ div } 5)$. Obtenemos un 2 como resultado. Resta calcular $2 + 1$, obteniendo un 3 como resultado final de la expresión. Ese 3 es el valor que asignamos al dato "c" que está a la izquierda de la flecha.

Con los contenidos vistos hasta el momento, ¡ya podemos plantear algoritmos para resolver algunos problemas muy sencillos! Por ejemplo:

- 1) Escribir un algoritmo para calcular el perímetro de un cuadrado a partir de una longitud de lado dada en cm.

Algoritmo `calculoDePerimetroDeCuadrado`

DE: `longLado` (entero) --- esta es la información con la que contamos al momento de iniciar las acciones

DS: `perimetro` (entero)

`perimetro` $\leftarrow$ `longLado` * 4

- 2) Escribir un algoritmo para calcular los centímetros correspondientes a una cantidad dada de metros.

Algoritmo `pasoAcm`

DE: `metros` (entero) --- esta es la información con la que contamos al momento de iniciar las acciones

DS: `cm` (entero)

`cm` $\leftarrow$ `metros` * 1000

Vamos a hacer algunas pruebas con “`calculoDePerimetroDeCuadrado`”. En la definición del algoritmo escrita en pseudo-código vemos una definición general. El paso siguiente que vamos a ir incorporando con la práctica consiste en hacer pruebas de nuestro algoritmo, para verificar su funcionamiento. Estas pruebas se llamarán TRAZAS, y son una herramienta fundamental en el proceso de desarrollo de algoritmos. Empecemos con algunas pruebas de manera informal.

Como vemos, en el algoritmo “`calculoDePerimetroDeCuadrado`” tenemos un único dato de entrada: la longitud del lado, llamado “`longLado`”. Vamos a hacer pruebas con distintos valores que elegimos nosotros para esa longitud de lado, y veremos el resultado que arrojan. Cada una de esas pruebas estará “simulando” una ejecución de nuestro algoritmo. Más adelante también analizaremos en detalle esa elección “random” que hacemos para los datos de entrada en nuestra verificación.

	DE	Acción	DS
Prueba 1	<code>longLado</code> vale 10	$\text{perimetro} \leftarrow \text{longLado} * 4$ Etapa 1: evaluar (<code>longLado</code> * 4) Como <code>longLado</code> vale 10, da como resultado 40. Etapa 2: asignar 40 al dato <code>perimetro</code>	<code>perimetro</code> vale 40
Prueba 2	<code>longLado</code> vale 2	$\text{perimetro} \leftarrow \text{longLado} * 4$ Etapa 1: evaluar (<code>longLado</code> * 4) Como <code>longLado</code> vale 2, da como resultado 8. Etapa 2: asignar 8 al dato <code>perimetro</code>	<code>perimetro</code> vale 8

Estos dos ejemplos son muy básicos. Incluyen solo una acción (la asignación), con lo que esta es la forma más simple que podemos ver de algoritmos completos. Sin embargo, nos sirve para reforzar que en nuestros algoritmos siempre vamos a tener que identificar claramente tres partes: cuáles son los **datos de entrada**, cuáles son los **datos de salida** y qué **acciones** nos permiten obtener los datos de salida a partir de la información contenida en los datos de entrada.

Además, como dijimos previamente, las asignaciones constituyen la principal acción de nuestros algoritmos ya que son las que nos permiten modificar los valores de los datos. Sin embargo, las mismas se irán combinando con otras acciones y entre ellas mismas, para generar

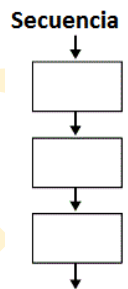
procesos más complejos. En este contexto, para **organizar las acciones**, surgen en la programación estructurada las **estructuras de control**.

Las estructuras de control son el conjunto de reglas que permiten controlar el flujo de ejecución de las acciones de un algoritmo (o de un programa). La mayoría de los lenguajes de programación actuales soportan o utilizan las mismas estructuras de control o, al menos, son muy parecidas. Lo que varía entre uno y otro es principalmente su sintaxis. Por esta razón veremos en lenguaje de diseño o pseudo-código las estructuras de control más comunes. Éstas te servirán de base para el aprendizaje de las estructuras de control en cualquier lenguaje de programación.

4.1 SECUENCIA

Esta regla indica que las acciones (instrucciones) se ejecutan sucesivamente, en el orden que se encuentran de arriba hacia abajo una a continuación de otra, sin posibilidad de omitir ninguna y sin bifurcaciones. Es decir que la acción k no se inicia hasta haber terminado la acción $k-1$.

Vamos a escribir un algoritmo que contenga acciones en secuencia. Hasta el momento la única acción que aprendimos fue la asignación. Entonces vamos a escribir un algoritmo “de juguete” con el único fin de explicar qué significa que las acciones se ejecutan de manera secuencial. Supongamos que nuestro algoritmo de juguete, al que llamaremos “ejemploSec1”, recibe un dato “a” que es un número, y retorna un dato “b” que es otro número. Y supongamos que se define con las siguientes acciones:



Algoritmo ejemploSec1

DE: a (entero)

DS: b (entero)

$b \leftarrow a + 10$

$a \leftarrow b * b$

$b \leftarrow a$

Ahora sí que se puso interesante... Vamos a verificar el funcionamiento de este algoritmo para algunos posibles valores del dato de entrada [a]. Para esto vamos a realizar una traza más parecida a lo que serán nuestras trazas finales. En la misma, vamos a colocar una columna por cada dato (no importa si es dato de entrada o dato de salida). Como tenemos un único dato de entrada y un único dato de salida (dos datos en total), vamos a preparar dos columnas. Será inicialmente algo así:

a	b

Una tabla igual que ésta se irá completando simulando en cada caso, ejecuciones independientes de nuestro algoritmo. Cada traza [tabla completa] representará un caso particular del problema que estamos resolviendo. Para esto, en cada “simulacro” le daremos distintos valores al (o a los) datos de entrada. Por ejemplo, comencemos con un caso donde nuestro dato a vale 10. Esto es lo que haremos cada vez que vamos a verificar nuestro algoritmo, pensamos nosotros con qué valor para los datos de entrada lo queremos probar.

Entonces el primer paso es poner en la tabla el valor del **dato de entrada** con el que elegimos verificar el algoritmo.

a	b
10	

Algoritmo ejemploSec1
 DE: a (entero)
 DS: b (entero)
 $b \leftarrow a + 10$
 $a \leftarrow b * b$
 $b \leftarrow a$

Recién ahora, una vez que determinamos con que valor de dato de entrada vamos a probar el algoritmo, empezamos a simular la ejecución de las acciones. Tenemos tres asignaciones y como dijimos, se deben ejecutar en secuencia, es decir, debemos respetar exactamente el orden de las mismas. Por esto, primero ejecutaremos la asignación $b \leftarrow a + 10$. Esta acción se lleva a cabo en dos etapas, la primera es la evaluación de la expresión que está a la derecha de la flecha. En este caso, $a + 10$. Dado que el dato **a** tiene valor 10 (como vemos en la última versión que tenemos de la tabla), la evaluación de esa expresión da 20. Una vez obtenido el valor de evaluar la expresión de la derecha, asignamos ese valor al nombre de dato de la izquierda, en este caso **b**. Luego de la ejecución de la primera acción de nuestro algoritmo, la tabla queda así:

a	b
10	20

Vamos a la segunda acción, que es la asignación $a \leftarrow b * b$. Nuevamente para ejecutarla tenemos que llevar a cabo dos tareas: evaluar la expresión de la derecha y asignar el valor obtenido al dato que aparece a la izquierda. La expresión es $b * b$. Para ver cuánto vale **b** en este momento de la ejecución tenemos que mirar la tabla. Nuestra tabla indica que **b** vale 20. Entonces para evaluar $b * b$ vamos a calcular $20 * 20$. El resultado es 400, ese mismo es el valor que tenemos que asignar al dato que aparece en el lado izquierdo de la flecha, que es **a**. Esto lo indicamos en la tabla de la siguiente manera:

a	b
10 400	20

Así indicamos que el nuevo valor de **a** es 400. Ya no vale 10. Cada dato tiene un único valor en cada instante de tiempo. Ahora vamos a ejecutar la tercera acción, que es la asignación $b \leftarrow a$. En este caso tenemos que del lado derecho solo tenemos que averiguar el valor de un dato, el dato **a**. Ese valor lo tenemos en la tabla y es 400. Entonces solo resta asignarlo directamente al dato que aparece a la izquierda que es **b**. Así quedaría nuestra traza luego de ejecutada esta asignación:

a	b
10 400	20 400

Como resultado de esta simulación de nuestro algoritmo, podemos decir que cuando el dato de entrada **a** vale 10, el dato de salida **b** vale 400. En este contexto, lo importante es entender que si cambiamos el orden de las acciones, el resultado cambia.

✎ Ahora te propongo lo siguiente. Para cada uno de los algoritmos que te dejo abajo, donde solo cambia el orden de las tres asignaciones, hacé una traza considerando que el valor del dato de entrada **a** nuevamente es 10. ¿Qué conclusiones podés sacar? ¡Consultame si te quedan dudas o acercate y mostrame los resultados que obtuviste! (Debería surgir al menos una duda importante de la resolución de esta actividad)

Algoritmo ejemploSec2DE: **a** (entero)DS: **b** (entero) $b \leftarrow a + 10$ $b \leftarrow a$ $a \leftarrow b * b$ **Traza ejemploSec2 con a=10**

a	b

Algoritmo ejemploSec3DE: **a** (entero)DS: **b** (entero) $b \leftarrow a$ $a \leftarrow b * b$ $b \leftarrow a + 10$ **Traza ejemploSec3 con a=10**

a	b

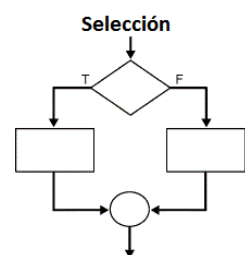
Algoritmo ejemploSec4DE: **a** (entero)DS: **b** (entero) $a \leftarrow b * b$ $b \leftarrow a + 10$ $b \leftarrow a$ **Traza ejemploSec4 con a=10**

a	b

4.2 CONDICIONAL, CAMINOS ALTERNATIVOS DE ACCIÓN

Muchas veces la ejecución de una o varias acciones dependerá de diferentes criterios. Para esos casos contamos con dos tipos de estructuras de control: la estructura de control condicional y la estructura de control para caminos alternativos no condicional.

Como la palabra lo indica, la estructura de control condicional es aquella que nos permite decidir si llevamos a cabo una determinada acción a partir de una condición, es decir, a partir de la respuesta a una pregunta de tipo "sí/no". El ejemplo en la vida cotidiana sería decir, "si nos juntamos el sábado, llevo el postre". Esto está indicando que la acción de llevar el postre está supeditada a que se cumpla la condición: "nos juntamos el sábado". Si "nos juntamos el sábado" resulta ser verdadero, entonces la acción "llevo el postre" se podrá ejecutar. La condición es "nos juntamos el sábado" (será verdadero o falso) y la acción es "llevo el postre". Es la primera forma de selección de caminos alternativos que veremos en esta sección.



SI

Las estructuras de control condicionales permiten modelar situaciones en las cuales la ejecución de una acción, o de un bloque de acciones, depende de una condición. Utilizamos el condicional cuando:

- dos acciones son mutuamente excluyentes según una condición.
- una acción depende de una condición,

Usando pseudo-código, las dos alternativas se escriben de la siguiente manera:

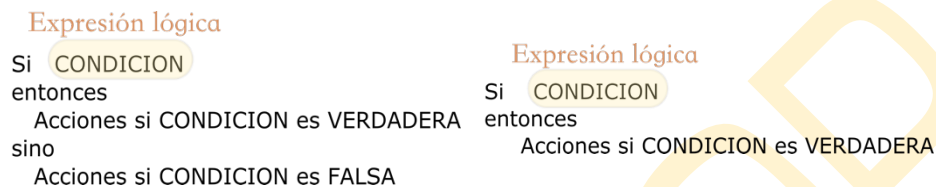


Figura 15. Estructura de control **CONDICIONAL**

En el primer caso podemos ver que tenemos **dos caminos posibles**. De estos dos caminos, sí o sí tomaremos alguno de ellos. Esto depende del valor lógico de la condición:

- Si el valor lógico de la condición es VERDADERO, tomamos el camino de acciones que depende de la palabra “entonces”.
- Si el valor lógico de la condición es FALSO, tomamos el camino de acciones que depende de la palabra “sino”.

En el segundo caso tenemos un camino que puede tomarse o no. Este camino sería como un camino opcional. Serán ejecutadas las acciones que dependen de la palabra “entonces” solo en el caso que la condición sea evaluada a un valor VERDADERO. En caso que la condición sea FALSA, ese camino no se elige, y el SI no tiene efecto.

Podemos visualizar cómo funciona esta dinámica en ejecución con los siguientes diagramas de flujo:

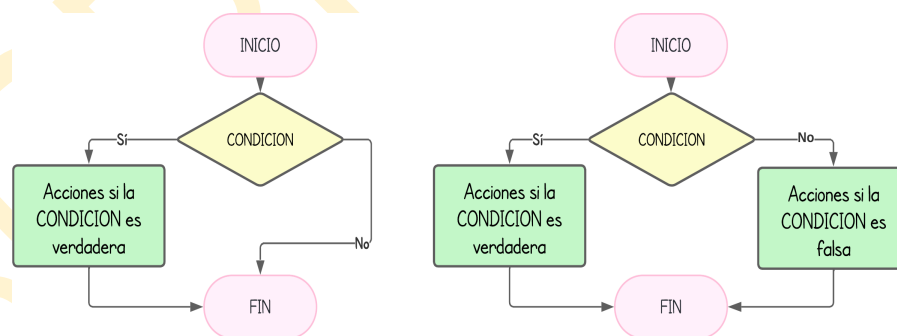


Figura 16. Diagrama de flujo de la estructura de control **CONDICIONAL**

Vamos a ver algunos ejemplos. Supongamos una situación de la vida real, donde tenemos acciones en secuencia y una acción que depende de una condición.

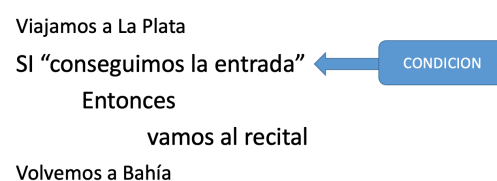


Figura 17. Ejemplo de estructura de control con un camino condicional

En este pseudo-algoritmo podemos ver que tenemos tres acciones. Las acciones son “Viajamos a La Plata”, “vamos al recital” y “Volvemos a Bahía”. Según este código, las acciones de viajar a La Plata y de volver a Bahía se harían siempre, es decir, son incondicionales. Sin embargo, podemos ver que la acción de “vamos al recital” es una acción que no se va a poder hacer siempre, sino que DEPENDE DE UNA CONDICIÓN. ¿Cuál es la condición? Si “conseguimos la entrada”. Esta condición es una circunstancia que, llegado el momento, se cumplirá o no se cumplirá, y de acuerdo a esto, estará supeditada la ejecución de la acción de ir al recital. Este primer pseudo-código de un caso de uso del condicional donde el camino especificado por el SI es *optativo*, puede ser representado con un diagrama de flujo:

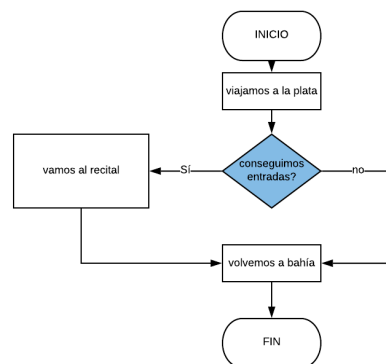


Figura 18. Ejemplo de estructura de control con un camino condicional: diagrama de flujo

Es importante aquí prestar atención a las acciones que se encuentran antes y después del SI. Como vemos, la estructura de control que guía siempre el orden es la secuencia, por lo que antes del condicional, siempre ejecutaremos la acción “Viajamos a La Plata”, y luego del SI, indistintamente de lo que suceda en el condicional, ejecutaremos la acción “Volvemos a Bahía”. Esto nos lleva a dos escenarios posibles:

Escenario posible 1	Escenario posible 2
Viajamos a La Plata Vamos al recital Volvemos a Bahía	Viajamos a La Plata Volvemos a Bahía

Supongamos ahora que aprovechamos las dos ramas de nuestro condicional. Lo hacemos con el siguiente ejemplo, viendo a la vez el pseudo-código y el diagrama de flujo correspondiente:

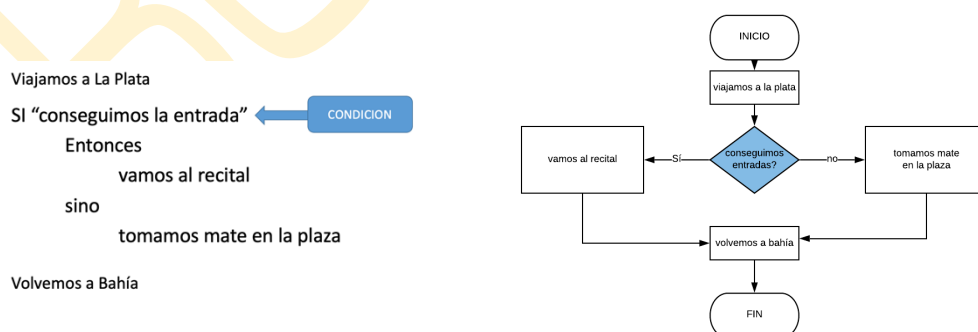


Figura 19. Ejemplo de estructura de control con dos caminos mutuamente excluyentes a partir de una condición

En este algoritmo podemos observar que hay cuatro acciones. “Viajamos a La Plata”, “Vamos al recital”, “Tomamos mate en la plaza”, y “Volvemos a Bahía”. Por la manera en la que fue escrito, detectamos que “conseguimos la entrada” es una CONDICIÓN, es decir, es una posibilidad que en el momento que se ejecute el algoritmo, se decidirá si es verdadera o falsa. De esta forma ahora quedan definidos dos posibles escenarios de ejecución, determinados por los dos posibles valores de la CONDICIÓN:

Escenario posible 1	Escenario posible 2
Viajamos a La Plata Vamos al recital Volvemos a Bahía	Viajamos a La Plata Tomamos mate en la plaza Volvemos a Bahía

Nuevamente es importante resaltar que las acciones que se encuentran en secuencia, antes y después del SI, se ejecutan indistintamente siempre más allá de lo que suceda dentro del condicional. Más adelante veremos que los condicionales se pueden anidar y combinar con el resto de las estructuras de control libremente.

Por último, veamos un algoritmo más cercano a los de RPA. Supongamos que queremos resolver el problema de determinar cuál es el mayor de dos números enteros. ¿Cuáles serían los datos de entrada? ¿Y los datos de salida? La información que deberíamos recibir son los valores de los dos números que queremos analizar, y podríamos retornar un dato que se llame “elMAYOR” que contenga el valor más grande. También podríamos pensar otras formas, pero de momento nos quedamos con esta. Vamos a escribir el algoritmo:

Algoritmo máximoDeDos

DE: $n1, n2$ (entero)

DS: $eIMAYOR$ (entero)

Si ($n1 > n2$)

entonces

$eIMAYOR \leftarrow n1$

sino

$eIMAYOR \leftarrow n2$

Como podemos ver, tenemos definidas dos asignaciones. Sin embargo tenemos que tener claro que en cada ejecución, solo una se va a llevar a cabo. Y esto depende del valor de verdad que obtenemos al evaluar la expresión “ $n1 > n2$ ”. ¿Qué sucede en nuestro algoritmo si los dos números $n1$ y $n2$ son iguales? Te lo dejo para pensar...

Pero en este punto hay algo que es muy importante recalcar: **el dato de salida SIEMPRE TIENE QUE TENER VALOR al finalizar la ejecución de nuestro algoritmo**. Es decir, el siguiente algoritmo es INCORRECTO, ya que en los casos que $n1$ es menor o igual a $n2$, el dato de salida $eIMAYOR$ no tiene valor.

Algoritmo máximoDeDosCONerror

DE: $n1, n2$ (entero)

DS: $eIMAYOR$ (entero)

Si ($n1 > n2$)

entonces

$eIMAYOR \leftarrow n1$

EN CASO DE

En todos los lenguajes de programación, así como vamos a encontrar una forma de escribir un condicional donde las acciones dependen del resultado de una expresión lógica, también vamos a tener la posibilidad de definir múltiples caminos de acción. ¿Cuántos caminos posibles tenemos usando un SI? Como vimos, pueden definirse máximo dos caminos mutuamente excluyentes (uno para el caso de que la condición sea verdadera, y otro camino para la condición falsa). Bueno, en todos los lenguajes vas a tener la posibilidad también de definir múltiples [más de dos] caminos. En algunos lenguajes esta estructura se llama “case”, en otros “switch”. Así como en el SI el camino elegido dependía del resultado de la evaluación de una expresión lógica, en este caso el camino elegido dependerá del valor de un dato, que en principio consideraremos que solo puede ser un número entero o un carácter. En lenguaje de diseño vamos a escribir esta estructura de control de la siguiente manera, en su forma más simple:

Dato de tipo entero o carácter

En caso de que el dato valga:

valor1: acciones1
valor2: acciones2
valor3: acciones3

Figura 20. Estructura de control para múltiples caminos alternativos mutuamente excluyentes

La lista de posibles valores que determina el camino que se va a seguir puede tener todas las opciones que necesitemos, e incluso puede contener valores alternativos para el mismo camino y también, intervalos de valores. Además, existe en todos los lenguajes una opción de dar un camino “por defecto” para el caso en que el dato no vale ninguno de los valores de la lista.

Veamos el siguiente ejemplo. Los turnos para vacunación en una salita médica se organizan según el último dígito del número de documento de la siguiente manera:

Ultimo dígito	Día para vacunarse
0 o 9	LUNES
1 al 5	MARTES
6	MIÉRCOLES
7	JUEVES
8	VIERNES

Queremos hacer una aplicación que le pida al usuario su número de documento y le indique el día que le corresponde vacunarse de acuerdo al último dígito del mismo.

Primero debemos analizar cuál o cuáles serían los datos de entrada. Como nos indica el enunciado, el usuario tiene que proveer su número de documento, por lo que el dato de entrada puede ser el número entero correspondiente a su DNI. Como dato de salida tendríamos un cartel por pantalla, ya que no se devuelve un resultado específico en un dato. Entonces podemos ahora escribir el algoritmo:

Algoritmo TurnosVacunaPorDNI
DE: DNI (un número entero)
DS: UN CARTEL indicando el día de la semana que se puede vacunar
Comienzo
 Leer(DNI)
 Obtener el último dígito del número DNI
 En caso que el último dígito valga:
 0 o 9: mostrar(LUNES)
 entre 1 y 5: mostrar(MARTES)
 6: mostrar(MIÉRCOLES)

7: mostrar(JUEVES)
8: mostrar(VIERNES)

Fin

En este algoritmo tenemos varias cosas para resaltar. La primera es que ya podemos ver dos acciones que no habíamos usado. Estas acciones son leer y mostrar. Cuando decimos “leer[DNI]” estamos indicando que el usuario ingresará un valor y ese valor lo tomaremos y asignaremos al dato DNI. Cuando decimos “mostrar[MENSAJE]” estamos indicando que ese mensaje será mostrado de alguna manera al usuario.

Luego podemos resumir tres las acciones que aprendimos hasta el momento de la siguiente forma:

- ASIGNACION
- Leer[DATO]
- Mostrar[MENSAJE]

Vamos también a aprovechar este ejemplo para introducir un concepto nuevo, el **DATO AUXILIAR**. Hay problemas en los que podemos utilizar datos que no son datos ni de entrada ni de salida, por ejemplo, para hacer cálculos parciales. En el ejemplo que estamos desarrollando, podemos usar un dato auxiliar para guardar el último dígito del DNI. Esto se haría de la siguiente manera:

Algoritmo TurnosVacunaPorDNI
DE: DNI (un número entero)
DS: UN CARTEL indicando el día de la semana que se puede vacunar
Dato Auxiliar: ultDigito (un número natural)
Comienzo
Leer(DNI)
ultDigito ← DNI mod 10
En caso que el (ultDigito) valga:
 0 o 9: mostrar(LUNES)
 entre 1 y 5: mostrar(MARTES)
 6: mostrar(MIERCOLES)
 7: mostrar(JUEVES)
 8: mostrar(VIERNES)
Fin

Como podemos ver, el trabajo de escribir algoritmos es un proceso en el que no sale de una vez la versión final. En cambio, muchas veces vamos a ir mejorando, o haciendo pequeñas modificaciones en el texto. A esto se lo denomina la estrategia de refinamiento paso a paso. Es ir detallando cada vez más los pasos hasta llegar a la versión final. Puede llevar varios cambios hasta llegar al algoritmo que finalmente pasaremos a nuestro lenguaje de programación.

Veamos otro ejemplo con un pequeño cambio en el enunciado. Supongamos ahora que los turnos para vacunación se organizan según la primera letra del apellido de la siguiente manera:

Primera letra del apellido	Día para vacunarse
A a la C	LUNES
D a la J	MARTES
K a la M	MIERCOLES
N a la R	JUEVES
S a la Z	VIERNES

En este caso cambia el dato de entrada. El usuario debe brindarnos la primera letra de su apellido, y de acuerdo a ella le diremos el día que le toca vacunarse. De esta forma obtenemos el siguiente algoritmo:

```

Algoritmo TurnosVacunaPorInicialDeApellido
DE: inicial (un caracter entra A y Z)
DS: UN CARTEL indicando el día de la semana que se puede vacunar
Comienzo
Leer(inicial)
En caso que la inicial valga:
    entre A y C: mostrar(LUNES)
    entre D y J: mostrar(MARTES)
    entre K y M: mostrar(MIERCOLES)
    entre N y R: mostrar(JUEVES)
    entre S y Z: mostrar(VIERNES)
Fin
  
```

De esta forma pudimos introducir las dos formas más comunes para establecer **caminos alternativos** de acción, los cuales son siempre mutuamente excluyentes. En el primer caso (del SI), el camino elegido depende del valor lógico de una condición [expresión lógica]. En el segundo caso (del EN CASO DE), el camino elegido depende del valor de un dato, que puede ser un número entero o un caracter.

✿ Ahora te propongo lo siguiente. Voy a escribir algoritmos para los que tenés que armar todos los posibles escenarios de acciones en ejecución.

Escenarios 1

```

Algoritmo escenarios1
DE: n1, n2 (entero)
DS: resultado (entero)
Dato auxiliar: Aux
Aux ← n1 + n2
Si (Aux > 0)
    entonces
        mostrar("Resultado POSITIVO")
    sino
        mostrar("Resultado es CERO o NEGATIVO")
  
```

Escenarios posibles:

Escenarios 2

```

Algoritmo escenarios2
DE: a (entero)
DS: resto (entero)
resto ← a MOD 3
Si (resto = 0)
entonces
    mostrar("El dato de entrada ES MULTIPLO de 3")
sino
    mostrar("El dato de entrada NO ES MULTIPLO de 3")
mostrar("ADIOS")

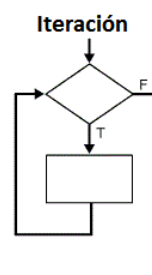
```

Escenarios posibles:

4.3 REPETICIÓN

Las estructuras de control repetitivas, también conocidas como bucles, son fundamentales en la programación para ejecutar acciones de manera reiterada. Estos bucles permiten automatizar procesos y reducir la redundancia de código. En la mayoría de los lenguajes de programación existen tres tipos principales de bucles: el bucle MIENTRAS, el bucle REPETIR-HASTA y el bucle PARA. Cada uno de estos bucles tiene sus propias características y se debe elegir a conciencia según las necesidades específicas del problema a resolver.

La cantidad de veces que se repiten las acciones puede quedar determinada por una variable de control que toma valores dentro de un rango escalar o por una expresión lógica. En el primer caso hablamos de iteración con contador y en el segundo hablamos de iteración con condición.



ITERACIÓN CON CONDICIÓN: MIENTRAS-HACER

Esta estructura de control permite repetir una o un grupo de acciones durante el tiempo que una determinada condición [expresión lógica] sea verdadera. Lo primero que hacemos al alcanzar una repetición del tipo "Mientras-hacer" es evaluar la CONDICION. Las acciones que dependen de esta estructura de control pueden no llegar a alcanzarse nunca en una ejecución. La forma general es:



Figura 21. Estructura de control REPETITIVA: mientras-hacer

La lógica de la misma es la siguiente: al llegar a este punto de nuestro algoritmo, lo primero que haremos es **evaluar la CONDICIÓN**. El resultado de esta evaluación, por ser una expresión lógica, puede ser verdadero o falso.

- Si el valor lógico de la CONDICIÓN es FALSO, no se ejecutará la sentencia contenida dentro de la estructura de control (las “Acciones si CONDICION es VERDADERA”). Se pasará directamente a la siguiente sentencia en secuencia luego del mientras.
- Si el valor lógico de la CONDICIÓN es VERDADERO, se ejecutará la sentencia interior del mientras (las “Acciones si CONDICION es VERDADERA”), y luego se volverá al inicio de la misma estructura de control. De manera repetida se irá evaluando la expresión y ejecutando la sentencia, mientras que la condición sea verdadera.

Para entender cómo funciona en ejecución podemos ver un diagrama de flujo:

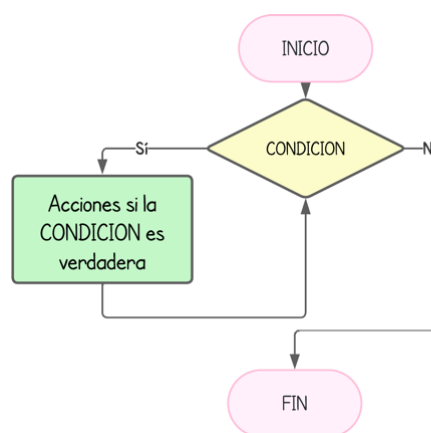


Figura 22. Diagrama de flujo de la estructura de control REPETITIVA: mientras-hacer.

Veamos un ejemplo. Vamos a suponer que queremos hacer un programa que muestre al usuario el doble de un número ingresado, para tantos números como decida el usuario, hasta que el usuario ingrese un cero indicando que no solicita más dobles. Un posible modelo de interacción sería el que se muestra a la derecha:

Este ejemplo nos permite ver distintas formas de resolución, las cuales serán todas igualmente correctas y dependerá del programador cuál será su manera preferida de hacerlo. Vamos a plantear una primera idea que iremos refinando. Podemos ver que hay un grupo de acciones que se podrían repetir muchas veces. Estas acciones son:

Pedir el ingreso de un número.
 Leer un número.
 Mostrar el doble del número ingresado.

Muy bien. Estas son las acciones que se deben repetir. Podemos empezar a escribir nuestro “mientras”.

Mientras (CONDICION) hacer
 Pedir el ingreso de un número.
 Leer un número.
 Mostrar el doble del número ingresado.

CONSOLA

```

Ingrese un número (termina con cero):
22
El doble es 44

Ingrese un número (termina con cero):
-131
El doble es -262

Ingrese un número (termina con cero):
0
FIN.-- ¡Gracias por usar mi programa!
  
```

Ahora tenemos que definir la **CONDICION**. Esta puede ser la parte más difícil. Sabemos que queremos repetir estas acciones mientras que el usuario ingrese números que sean distintos de cero. Entonces, qué tal si usamos un dato auxiliar, "distintoDeCero", que sea lógico, el cuál será verdadero solo en el momento en que el usuario ingrese un cero. Mientras tanto, este dato será falso. Vamos a ver cómo quedaría.

```

distintoDeCero ← VERDADERO
Mientras (distintoDeCero es VERDADERO) hacer
    Pedir el ingreso de un número.
    Leer un número.
    Si (el número es distinto de cero)
        Entonces
            Mostrar el doble del número ingresado.
        sino
            distintoDeCero ← FALSO

```

Si elegimos este enfoque, el algoritmo completo quedaría de la siguiente manera:

Algoritmo dobles1

DE: varios números hasta ingresar 0

DS: el doble de cada número ingresado que se muestra por pantalla

Daux: num (entero), distintoDeCero (lógico)

```

distintoDeCero ← VERDADERO
Mientras (distintoDeCero es VERDADERO) hacer
    Mostrar('Ingrese un número (termina con cero): ')
    Leer(num)
    Si (num <> 0)
        Entonces
            Mostrar(2 * num)
        sino
            distintoDeCero ← FALSO
Mostrar('Fin.-- ¡Gracias por usar mi programa!')

```

Sentencia compuesta

Otra forma que podríamos pensar es la siguiente. Podríamos comenzar pidiendo un número (ya sabemos que al menos un número el usuario deberá ingresar, que en el peor caso será 0 en la primera vez y no tendremos que mostrar ningún doble). En este caso tendríamos el siguiente algoritmo, donde en lugar de usar el dato lógico "distintoDeCero" directamente usamos el valor del dato "num" para armar la **CONDICION** y determinar si repetimos o no.

Algoritmo dobles2

DE: varios números hasta ingresar 0

DS: el doble de cada número ingresado que se muestra por pantalla

Daux: num (entero)

```

Mostrar('Ingrese un número (termina con cero): ')
Leer(num)
Mientras (num <> 0) hacer
    Mostrar(2 * num)
    Mostrar('Ingrese un número (termina con cero): ')
    Leer(num)
Mostrar('Fin.-- ¡Gracias por usar mi programa!')

```

Sentencia compuesta

Veamos algunos escenarios posibles en ejecución para esta versión de nuestra solución. Es importante remarcar que ya en este punto, los escenarios posibles son infinitos; tantos como posibles números que puede ingresar el usuario (¡incluyendo la posibilidad de que repita números!).

Escenario posible 1	Escenario posible 2	Escenario posible 3
Mostrar['Ingresa un núm... '] Leer(num) Mostrar[2 * num] Mostrar['Ingresa un núm... '] Leer(num) Mostrar[2 * num] Mostrar['Ingresa un núm... '] Leer(num) Mostrar['Fin...']	Mostrar['Ingresa un núm... '] Leer(num) Mostrar['Fin...']	Mostrar['Ingresa un núm... '] Leer(num) Mostrar[2 * num] Mostrar['Ingresa un núm... '] Leer(num) Mostrar['Fin...']

Como podemos observar, una característica importante de esta estructura de control es que puede suceder que las acciones que dependen del mientras **NUNCA SE EJECUTEN**. Esto sucede cuando la primera vez que evaluamos la **CONDICION**, obtenemos un resultado **FALSO**. Tal propiedad del mientras queda bien representada en el siguiente ejemplo de la vida cotidiana:

Hacer una llamada telefónica
 Marcar el número
Mientras teléfono ocupado
hacer
 Colgar
 Marcar el número
 Esperar a que atiendan
 Hablar

Aquí vemos claramente que un escenario posible es marcar el número y que el teléfono no de ocupado. En ese caso esperamos a que atiendan y hablamos. Solo para los casos donde después de marcar escuchemos tono de ocupado, deberíamos colgar y volver a marcar el número.

Como resumen podemos decir que:

- Las acciones del mientras-hacer pueden no ejecutarse nunca.
- Cuando la **CONDICION** se evalúa y da **FALSO**, se dice que se verifica la **condición de corte** (la que hace que se deje de repetir).
- En el bloque iterativo (acciones del mientras) se debe modificar al menos una de las variables involucradas en la **CONDICION** para permitir que la **condición de corte** se produzca.
- Si la condición de corte no se verifica nunca, se produce un **bucle infinito**.

EJEMPLO DE BUCLE INFINITO

Algoritmo **dobles2conBUCLEinfinito**

DE: varios números hasta ingresar 0

DS: el doble de cada número ingresado que se muestra por pantalla

Daux: num (entero)

Mostrar['Ingresa un número (termina con cero): ']
 Leer(num)

Mientras (num <> 0) hacer

Mostrar(2 * num)

Mostrar('Ingrese un número (termina con cero): ')

Mostrar('Fin.-- ¡Gracias por usar mi programa!')

☛ ¿Cómo imaginás que será la ejecución de este bucle? Asumiendo que se ingresa un 3 inicialmente... ¿te animás a ensayar una traza?

ITERACIÓN CON CONDICIÓN: REPETIR-HASTA

Esta estructura de control permite repetir una o un grupo de acciones durante el tiempo que una determinada condición (expresión lógica) es falsa. Cuando la condición pasa a ser verdadera, se dejan de repetir las acciones. Lo primero que hacemos al alcanzar un bucle del tipo "Repetir-hasta" es ejecutar sus acciones. Las acciones que dependen de esta estructura de control se ejecutan siempre por lo menos UNA VEZ. La forma general es:

Repetir
Acciones
hasta **CONDICION**
Expresión lógica

Figura 23. Estructura de control REPETITIVA: repetir-hasta

Y para entender mejor cómo funciona en ejecución podemos representarla con el siguiente diagrama de flujo:

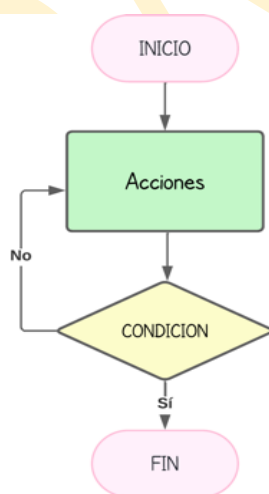


Figura 24. Diagrama de flujo de la estructura de control REPETITIVA: repetir-hasta

La principal diferencia que tenemos que notar entre esta estructura de control y la que vimos antes, es que en este caso, las acciones que dependen del *repetir* se ejecutan al menos una vez. En el caso del *mientras* podía suceder que no se ejecuten nunca.

Volvamos al ejemplo de la sección anterior donde queríamos pedir al usuario una cantidad variable de números enteros y mostrarle el doble de cada uno de ellos. El ingreso terminaba cuando el usuario ingresaba un cero. Para este problema donde sabemos que el usuario al menos ingresará un número (si no desea ver ningún doble, ingresará un cero), la estructura de control más adecuada será el repetir-hasta. Luego podemos escribir el siguiente algoritmo:

CONSOLA

```
Ingrese un número (termina con cero):
22
El doble es 44

Ingrese un número (termina con cero):
-131
El doble es -262

Ingrese un número (termina con cero):
0
FIN.-- ¡Gracias por usar mi programa!
```

Algoritmo dobles3

DE: varios números hasta ingresar 0

DS: el doble de cada número ingresado que se muestra por pantalla

Daux: num (entero)

Repetir

Mostrar('Ingrese un número (termina con cero): ')

Leer(num)

Si (num <> 0)

entonces

Mostrar(2 * num)

hasta (num = 0)

Mostrar('Fin.-- ¡Gracias por usar mi programa!')

Como se puede ver en el ejemplo, las acciones en este caso al menos una vez serán ejecutadas. Recién luego de ejecutar las acciones del repetir, se evalúa la condición. Si la misma es verdadera se sale del bucle. Si es falsa, se vuelven a ejecutar las acciones y de nuevo se evalúa la condición. En este caso nuevamente podemos resumir las siguientes características:

- Las acciones del repetir-hasta siempre se ejecutan al menos una vez.
- Cuando la CONDICION se evalúa y da VERDADERO, se dice que se verifica la **condición de corte** (la que hace que se deje de repetir).
- En el bloque iterativo (acciones del repetir) se debe modificar al menos una de las variables involucradas en la CONDICION para permitir que la **condición de corte** se produzca.
- Si la condición de corte no se verifica nunca, se produce un **bucle infinito**.

✎ Pensá un código sencillo con un repetir-hasta donde quede en evidencia un bucle infinito y escribilo acá abajo.

ITERACIÓN CON CONTADOR: PARA-DESDE-HASTA

En este punto lo primero que tenemos que remarcar es que esta estructura de control repetitiva es diferente a las dos anteriores. No solamente porque la cantidad de repeticiones no está determinada por la evaluación de una expresión lógica, sino también por la forma en que está implementada. Se podría decir que es una estructura de control más “sofisticada”, ya que tiene muchas características que tenemos que conocer que suceden “detrás de escena”. Vamos por partes. Primero veamos su forma general, y luego la vamos a ir analizando por partes:

Expresión Inicial Expresión Final
 Para VC desde EI hasta EF hacer
Variable Acciones
de Control

Figura 25. Estructura de control REPETITIVA: para-desde-hasta

Esta estructura de control nos permite, en su forma más básica, repetir N veces. Es decir, determinar si queremos repetir una cantidad de veces N, podríamos escribir:

Repetir N veces
 Mostrar('*')

Para VC desde 1 hasta N hacer
 Mostrar('*')

Este código repite N veces la acción de mostrar el asterisco. Es decir, que si N vale 3, se mostrarán 3 asteriscos; si N vale 10, se mostrarán 10 asteriscos. ¿Qué representa VC? VC será un identificador definido por nosotros, y representa la llamada “Variable de Control” del bucle. Este dato es muy importante y es manejado internamente por el lenguaje. Veremos más detalles en el próximo capítulo, pero en este punto lo que necesitamos entender es que ese dato va a recorrer el intervalo definido por la EI (Expresión Inicial) y la EF (Expresión Final) y en cada paso de ese recorrido, ejecutará las acciones del bucle. Es decir, en el ejemplo, el dato VC pasará por todos los enteros entre 1 y N, y en su paso por cada valor de ese intervalo ejecutará las acciones del bucle. Al tratarse de una estructura de control cuya forma de implementación está muy ligada al lenguaje utilizado, dejaremos los detalles para el próximo capítulo. Como conclusión final de este capítulo les dejo algunas trazas para repasar las estructuras de control vistas. Pueden hacer las trazas en una hoja aparte y luego controlar si el resultado que obtienen es el mismo que el mostrado.

REGISTRO DE TODO EL PROCESO DE CAMBIOS EN LOS VALORES DE LOS DATOS, EN DISTINTOS EJEMPLOS (CASOS DE TESTEO) SIMULANDO LA EJECUCIÓN.

Algoritmo T1

DE: -

DS: -

Dauxiliares: A, B (enteros); C (lógico)

comienzo

$A \leftarrow 10$

$B \leftarrow -20 + 2 * A$

$A \leftarrow A \text{ div } 4$

$C \leftarrow [A > B]$

$A \leftarrow [A * 11] \bmod 3$

fin

A	B	C
10	0	true
2		
1		

Algoritmo T2

DE: A (real)

DS: B (real), C (entero)

Dauxiliar: D (entero)

comienzo

$B \leftarrow A / 2$

$D \leftarrow \text{round}(A)$

$A \leftarrow A + 3$

$C \leftarrow \text{trunc}(B) + \text{trunc}(A) * D$

fin

// i. Con A valiendo 2.4

A	B	C	D
2.4	1.2	11	2
5.4			

// ii. Con A valiendo 5.7

A	B	C	D
5.7	2.85	50	6
8.4			

AGREGAMOS EL **EVALUADOR DE EXPRESIONES (EE)**: SE REGISTRAN LAS EVALUACIONES DE EXPRESIONES CONTENIDAS EN LAS ESTRUCTURAS DE CONTROL, LAS CUALES DETERMINAN EL CAMINO QUE SE SIGUE EN CADA CASO.

Algoritmo T3

DE: A (entero)

DS: B (entero)

comienzo

Si $[A > 10]$

Entonces

$B \leftarrow A$

Sino

$B \leftarrow 0$

fin

// i. Con A valiendo 15

A	B	EE:
15	15	$A > 10 \rightarrow 15 > 10 \rightarrow \text{true}$

// ii. Con A valiendo 2

A	B	EE:
2	0	$A > 10 \rightarrow 2 > 10 \rightarrow \text{false}$

Algoritmo T4

DE: A [entero]

DS: B [entero]

comienzo

$B \leftarrow A * 2$

Si $(B \bmod 3 = 0)$

Entonces

$B \leftarrow B + 1$

fin

// i. Con A valiendo 1

A	B	EE:
1	2	$B \bmod 3 = 0 \rightarrow 2 \bmod 3 = 0$ $\rightarrow 2 = 0 \rightarrow \text{false}$

// ii. Con A valiendo 12

A	B	EE:
12	24 25	$B \bmod 3 = 0 \rightarrow 24 \bmod 3 = 0$ $\rightarrow 0 = 0 \rightarrow \text{true}$

Algoritmo T5

DE: A [entero]

DS: B [entero]

comienzo

$B \leftarrow 0$

Mientras $(A > 0)$ hacer

$B \leftarrow B + A * A$

$A \leftarrow A - 1$

Fin mientras

fin

Con A valiendo 4

A	B	EE:
4	0	$A > 0 \rightarrow 4 > 0 \rightarrow \text{true}$
3	16	$A > 0 \rightarrow 3 > 0 \rightarrow \text{true}$
2	25	$A > 0 \rightarrow 2 > 0 \rightarrow \text{true}$
1	29	$A > 0 \rightarrow 1 > 0 \rightarrow \text{true}$
0	30	$A > 0 \rightarrow 0 > 0 \rightarrow \text{false}$



Algoritmo T6

DE: A [entero]

DS: B [entero]

comienzo

$B \leftarrow 1$

Mientras ($A > 0$) hacer

$B \leftarrow B + A$

Si ($B \bmod 3 = 0$)

entonces

$A \leftarrow A - 1$

Sino

$A \leftarrow A - 3$

Fin mientras

fin

Con A valiendo 8

A	B
8	4
7	9
4	16
1	20
0	21

EE:

$A > 0 \rightarrow 8 > 0 \rightarrow \text{true}$

$B \bmod 3 = 0 \rightarrow 9 \bmod 3 = 0 \rightarrow 0 = 0 \rightarrow \text{true}$

$A > 0 \rightarrow 7 > 0 \rightarrow \text{true}$

$B \bmod 3 = 0 \rightarrow 16 \bmod 3 = 0 \rightarrow 1 = 0 \rightarrow \text{false}$

$A > 0 \rightarrow 4 > 0 \rightarrow \text{true}$

$B \bmod 3 = 0 \rightarrow 20 \bmod 3 = 0 \rightarrow 2 = 0 \rightarrow \text{false}$

$A > 0 \rightarrow 1 > 0 \rightarrow \text{true}$

$B \bmod 3 = 0 \rightarrow 21 \bmod 3 = 0 \rightarrow 0 = 0 \rightarrow \text{true}$

$A > 0 \rightarrow 0 > 0 \rightarrow \text{false}$

Algoritmo T7

DE: A [entero]

DS: B [entero]

comienzo

$B \leftarrow 0$

Repetir

$B \leftarrow B + A * 2$

Hasta ($B > 10$)

fin

Con A valiendo 2

A	B
2	0
	4
	8
	12

EE:

$B > 10 \rightarrow 4 > 10 \rightarrow \text{false}$

$B > 10 \rightarrow 8 > 10 \rightarrow \text{false}$

$B > 10 \rightarrow 12 > 10 \rightarrow \text{true}$

APENDICE II La Tabla ASCII

Caracteres ASCII de control			Caracteres ASCII imprimibles				ASCII extendido									
00	NULL	(carácter nulo)	32	espacio	64	@	96	`	128	Ç	160	á	192	Ł	224	Ó
01	SOH	(inicio encabezado)	33	!	65	A	97	a	129	ú	161	í	193	ł	225	ô
02	STX	(inicio texto)	34	"	66	B	98	b	130	é	162	ó	194	Ł	226	Ô
03	ETX	(fin de texto)	35	#	67	C	99	c	131	â	163	ú	195	ł	227	Õ
04	EOT	(fin transmisión)	36	\$	68	D	100	d	132	ä	164	ñ	196	—	228	ö
05	ENQ	(consulta)	37	%	69	E	101	e	133	à	165	Ñ	197	†	229	Ö
06	ACK	(reconocimiento)	38	&	70	F	102	f	134	â	166	ª	198	ä	230	µ
07	BEL	(timbre)	39	'	71	G	103	g	135	ç	167	º	199	Å	231	þ
08	BS	(retroceso)	40	(	72	H	104	h	136	ê	168	¿	200	Ł	232	Ɔ
09	HT	(tab horizontal)	41	)	73	I	105	i	137	ë	169	©	201	Ł	233	Ú
10	LF	(nueva línea)	42	*	74	J	106	j	138	è	170	¬	202	Ł	234	Ù
11	VT	(tab vertical)	43	+	75	K	107	k	139	ï	171	½	203	Ł	235	Û
12	FF	(nueva página)	44	,	76	L	108	l	140	î	172	¼	204	Ł	236	ý
13	CR	(retorno de carro)	45	-	77	M	109	m	141	ï	173	ı	205	=	237	Ÿ
14	SO	(desplaza afuera)	46	.	78	N	110	n	142	Ä	174	«	206	Ł	238	—
15	SI	(desplaza adentro)	47	/	79	O	111	o	143	Å	175	»	207	Ł	239	'
16	DLE	(esc.vínculo datos)	48	0	80	P	112	p	144	É	176	×	208	ø	240	≡
17	DC1	(control disp. 1)	49	1	81	Q	113	q	145	æ	177	÷	209	Ð	241	±
18	DC2	(control disp. 2)	50	2	82	R	114	r	146	Æ	178	×	210	È	242	—
19	DC3	(control disp. 3)	51	3	83	S	115	s	147	ô	179	ı	211	Ê	243	¾
20	DC4	(control disp. 4)	52	4	84	T	116	t	148	ö	180	ı	212	Ë	244	ŧ
21	NAK	(conf. negativa)	53	5	85	U	117	u	149	ò	181	À	213	İ	245	§
22	SYN	(inactividad sinc)	54	6	86	V	118	v	150	ù	182	Â	214	Í	246	÷
23	ETB	(fin bloque trans)	55	7	87	W	119	w	151	û	183	Á	215	Î	247	°
24	CAN	(cancelar)	56	8	88	X	120	x	152	ÿ	184	Â	216	Ï	248	ˆ
25	EM	(fin del medio)	57	9	89	Y	121	y	153	Ö	185	Ë	217	Ĵ	249	˙
26	SUB	(sustitución)	58	:	90	Z	122	z	154	Ü	186	Ï	218	Œ	250	˘
27	ESC	(escape)	59	;	91	[	123	{	155	ø	187	Œ	219	Œ	251	˚
28	FS	(sep. archivos)	60	<	92	\	124		156	£	188	Œ	220	Œ	252	˚
29	GS	(sep. grupos)	61	=	93	]	125	}	157	Ø	189	Œ	221	Œ	253	˚
30	RS	(sep. registros)	62	>	94	^	126	~	158	×	190	Œ	222	Œ	254	˚
31	US	(sep. unidades)	63	?	95	_			159	f	191	Œ	223	Œ	255	nbsp
127	DEL	(suprimir)														